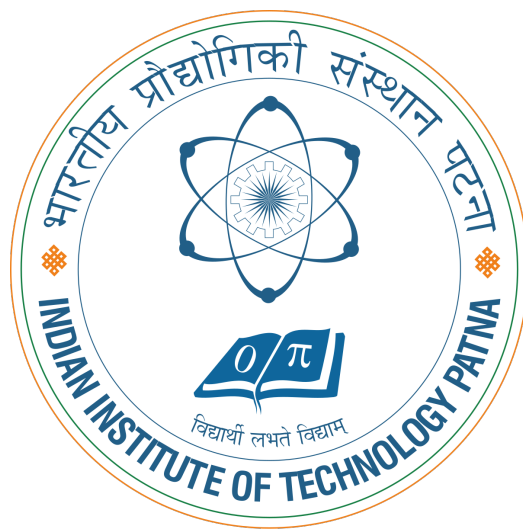


IMPLEMENTING LINEAR REGRESSION USING A MULTI-LAYER NEURAL NETWORK FROM SCRATCH



Ritik Yadav (2621MA03)

Sahnil Siddique (2621MA02)

Suman Acharya (2621MA11)

Department of Mathematics
Indian Institute of Technology Patna

1. Objectives

The objective of this task is to predict the median house value (MedHouseVal) from two input features, Median Income (MedInc) and House Age (HouseAge), using a multi-layer neural network implemented from scratch in Python. The required network contains two hidden layers with ReLU activation, followed by a single linear output neuron. The notebook also investigates learning-rate effects and compares Basic Gradient Descent, Momentum, and Adam optimizers.

2. Dataset and Preprocessing

The notebook loads the California Housing dataset using `fetch_california_housing` (`as_frame=True`). The resulting data contains 20,640 observations and 9 columns, including the two selected predictors MedInc and HouseAge and the target MedHouseVal. The selected input matrix has shape (20,640, 2), while the target has shape (20,640, 1).

A reproducible random seed of 42 is used before shuffling the observations. The data is split into 80% training and 20% testing, giving 16,512 training samples and 4,128 test samples. The two features are standardized using the training-set mean and standard deviation, and the same training statistics are then applied to the test set. The notebook reports approximately zero training means and unit training standard deviations after normalization.

Item	Value
Dataset	California Housing
Total observations	20,640
Input features	MedInc, HouseAge
Target	MedHouseVal
Training set	16,512 (80%)
Test set	4,128 (20%)
Random seed	42

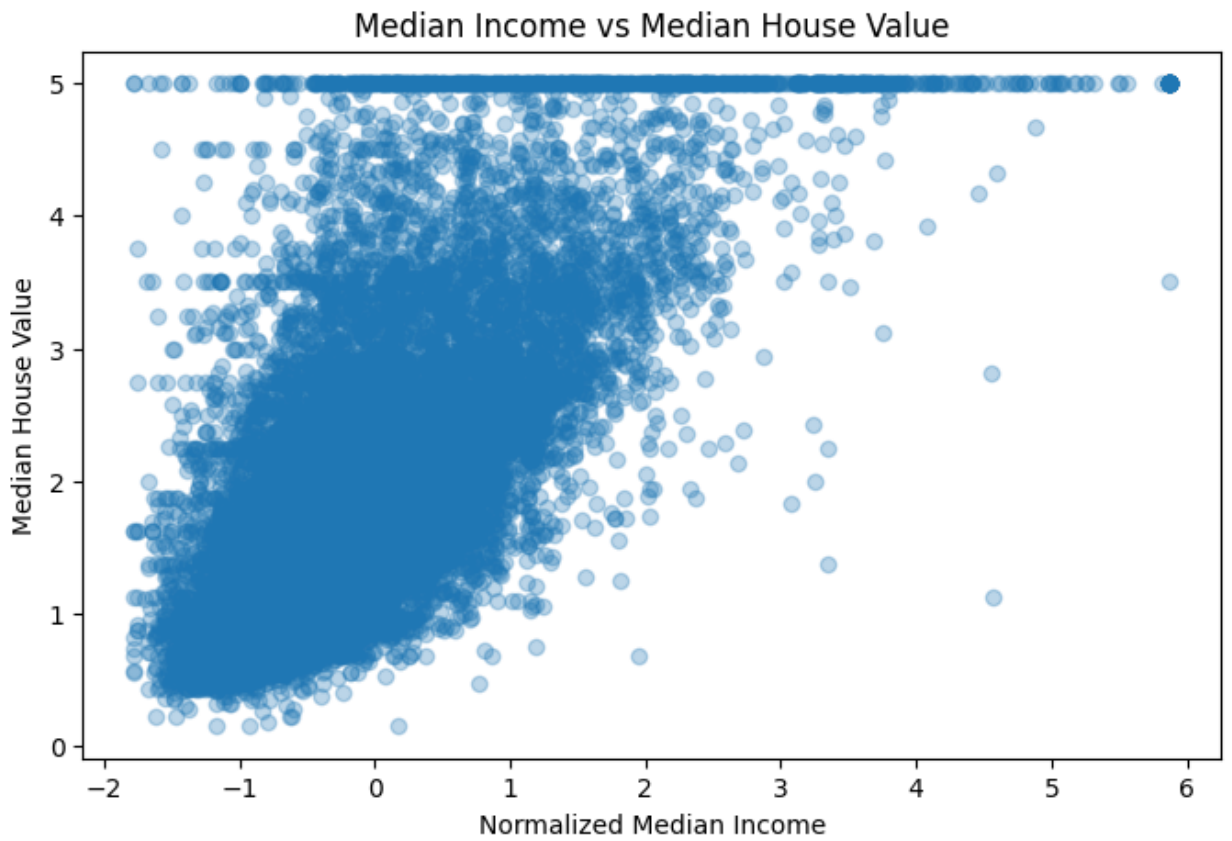


Figure 1: Normalized Median Income versus Median House Value.

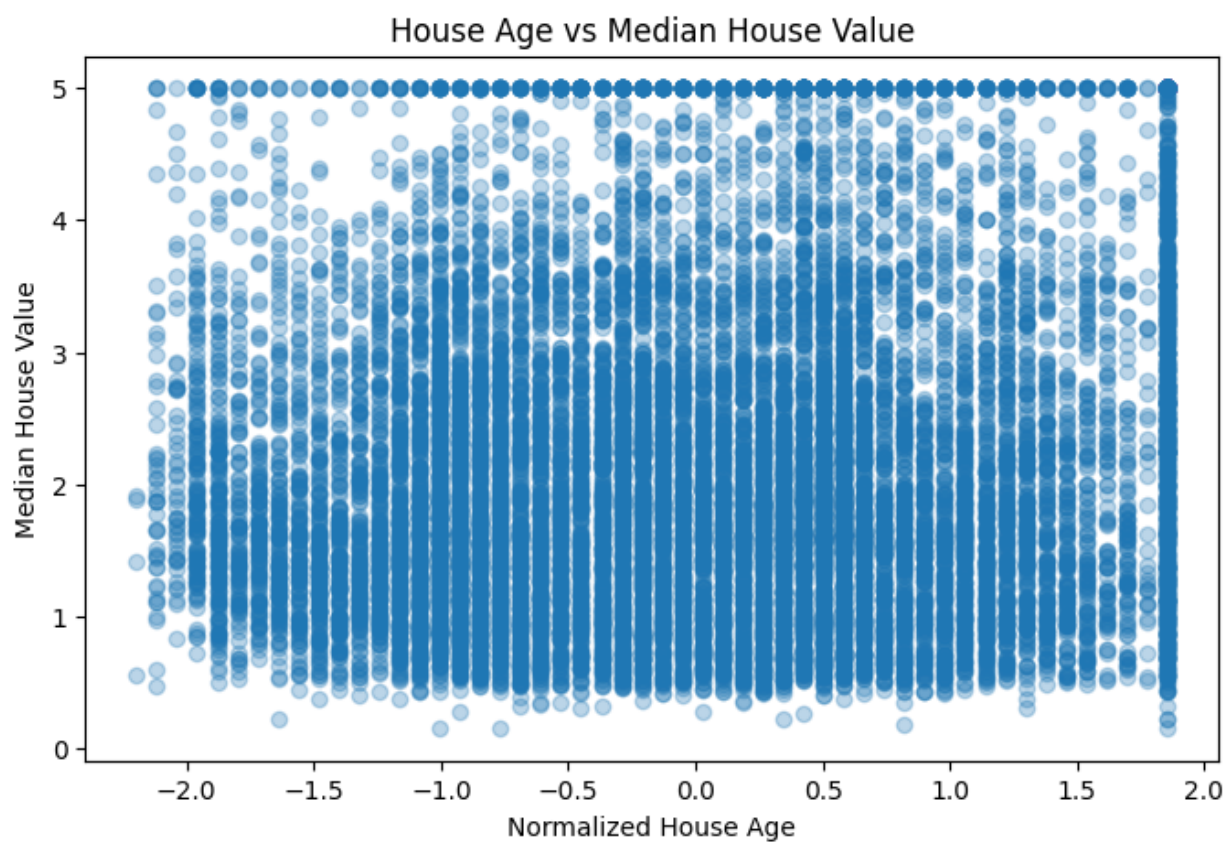


Figure 2: Normalized House Age versus Median House Value.

3. Neural Network Architecture

The implemented architecture follows the assignment specification exactly:

Layer	Neurons	Activation	Purpose
Input	2	—	MedInc and HouseAge
Hidden 1	5	ReLU	Non-linear representation
Hidden 2	3	ReLU	Further feature transformation
Output	1	Linear	Continuous MedHouseVal prediction

Forward Propagation

For the first hidden layer, $Z1 = XW1 + b1$ and $A1 = \text{ReLU}(Z1)$. For the second hidden layer, $Z2 = A1W2 + b2$ and $A2 = \text{ReLU}(Z2)$. The output layer computes $\hat{y} = A2W3 + b3$. The output layer is linear because the task is regression rather than classification.

Loss Function

Mean Squared Error (MSE) is used as the training and evaluation loss:

$$MSE = (1/n) \sum (y - \hat{y})^2$$

Backpropagation

The notebook computes analytical gradients through all three weight layers using the derivative of MSE and the derivative of ReLU. Parameters are then updated without using a pre-built neural-network model from TensorFlow, PyTorch, or another machine-learning framework.

Weight Initialization

Weights are initialized with scaled random normal values using a He-style initialization, while biases are initialized to zero. This is appropriate for the ReLU hidden layers.

4. Optimization Methods

Basic Gradient Descent: Parameters are updated directly in the opposite direction of their gradients using the selected learning rate.

Momentum: A velocity term is maintained for each parameter so that updates retain information from previous gradients. The notebook uses $\beta_1 = 0.9$.

Adam: First- and second-moment estimates are maintained for each parameter, followed by bias correction. The notebook uses $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1e - 8$, and a learning rate of 0.001.

5. Experimental Setup

The main optimizer experiments use 1,000 epochs. Basic Gradient Descent is tested with learning rates 0.01 and 0.001. Momentum is tested with learning rate 0.01, while Adam is tested with learning rate 0.001. Training loss is tracked throughout the 1,000 epochs.

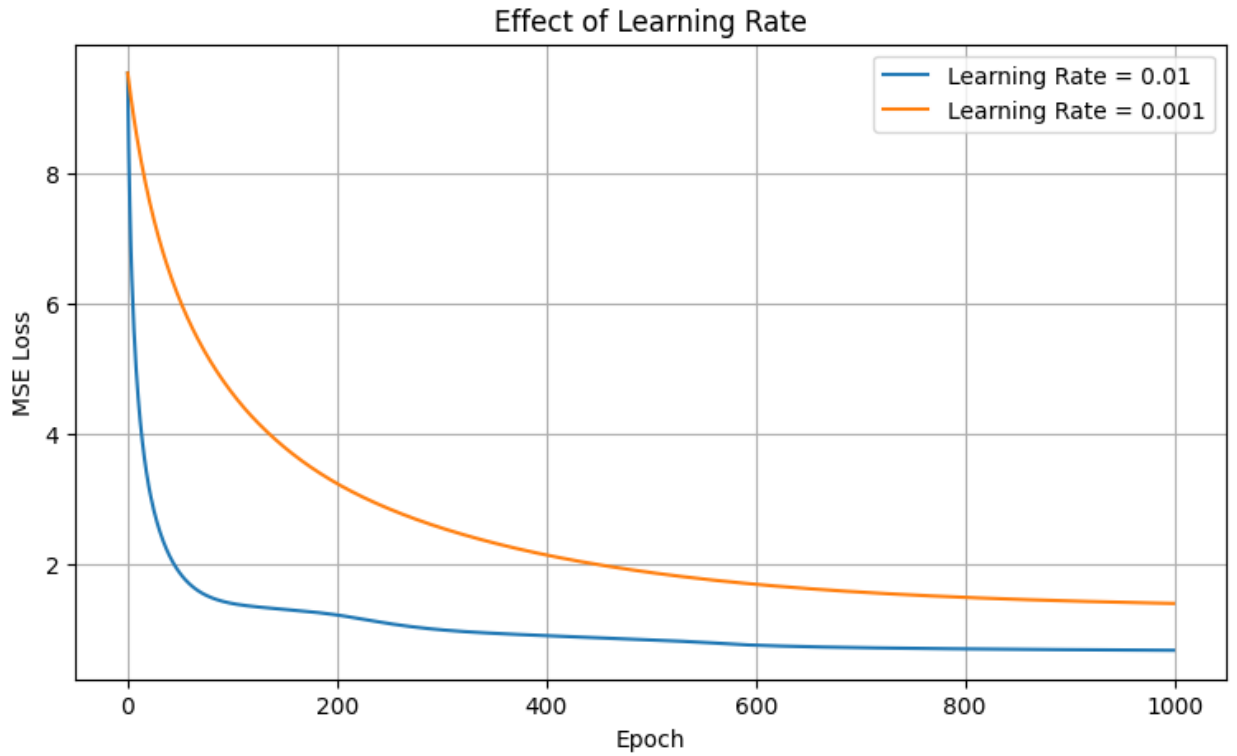


Figure 3: Training-loss comparison for learning rates 0.01 and 0.001.

6. Learning Rate Experiment

Learning Rate	Test MSE
0.01	0.698379
0.001	1.412956

With the recorded run, learning rate 0.01 performs substantially better after 1,000 epochs than 0.001. The test MSE is approximately 0.6984 for 0.01 versus 1.4130 for 0.001. The smaller learning rate makes much slower progress over the same fixed number of epochs.

7. Optimizer Comparison

Optimizer	Learning Rate	Test MSE	Test RMSE
Gradient Descent	0.010	0.698379	0.835691
Momentum	0.010	0.702638	0.838235
Adam	0.001	0.695869	0.834188

Among the three recorded optimizer runs, Adam achieves the lowest test MSE (0.695869) and the lowest test RMSE (0.834188). Basic Gradient Descent is very close, with a test MSE of 0.698379. Momentum gives the highest test MSE of the three runs at 0.702638. The differences are relatively small, so all three methods reach broadly similar final predictive performance under the selected settings.

Training-loss checkpoints

Run	Epoch 1	Epoch 500	Epoch 1000
GD, Ir=0.01	9.531136	0.834348	0.674503
GD, Ir=0.001	9.531136	1.869019	1.393822
Momentum, Ir=0.01	9.531136	0.885095	0.679411
Adam, Ir=0.001	9.531136	0.742752	0.666955

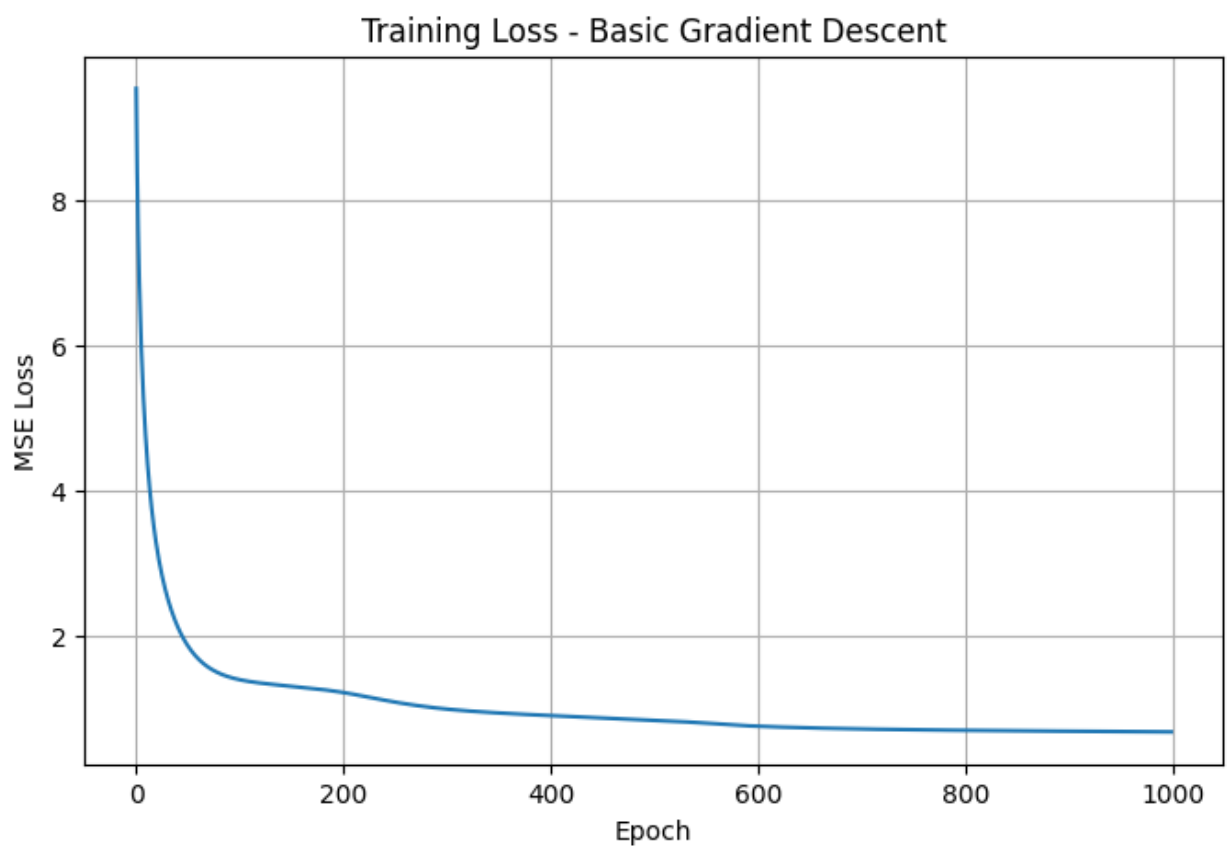


Figure 4: Training loss for Basic Gradient Descent with learning rate 0.01.

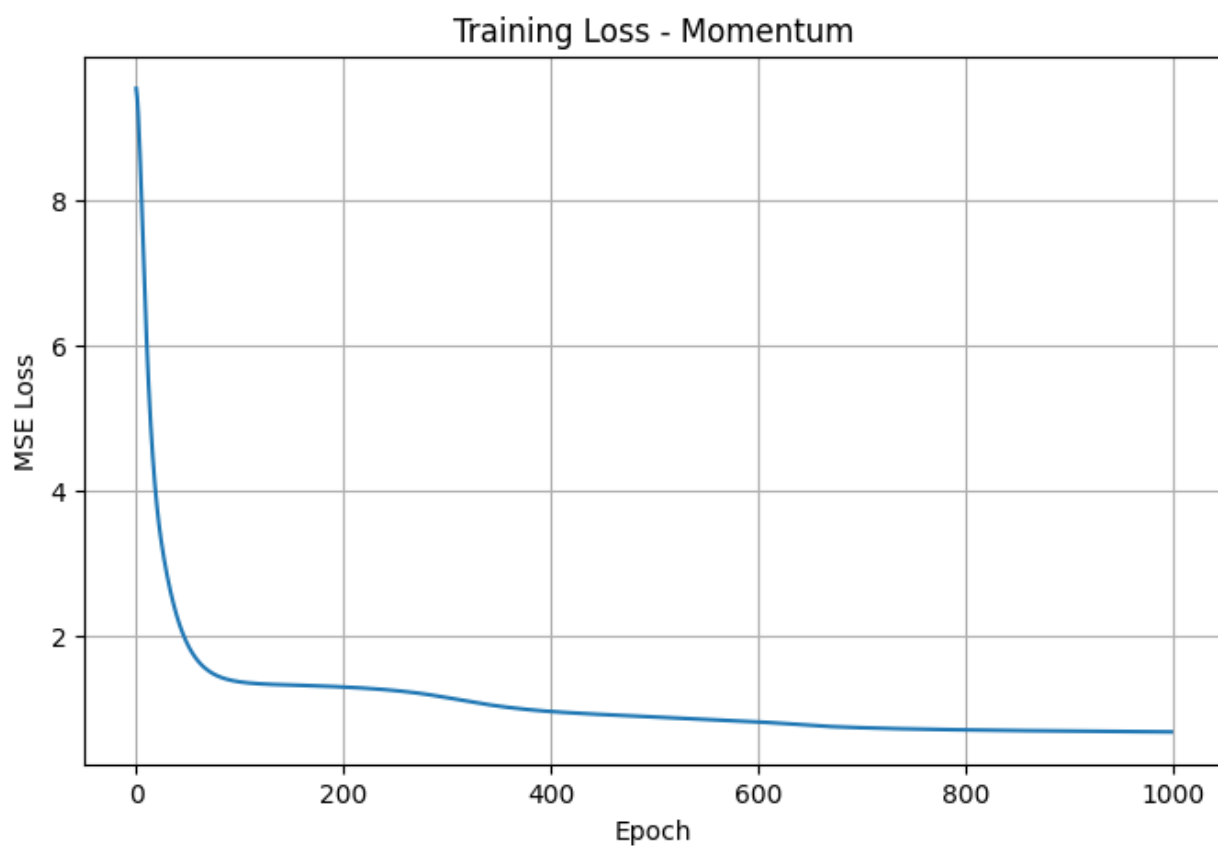


Figure 5: Training loss for Momentum.

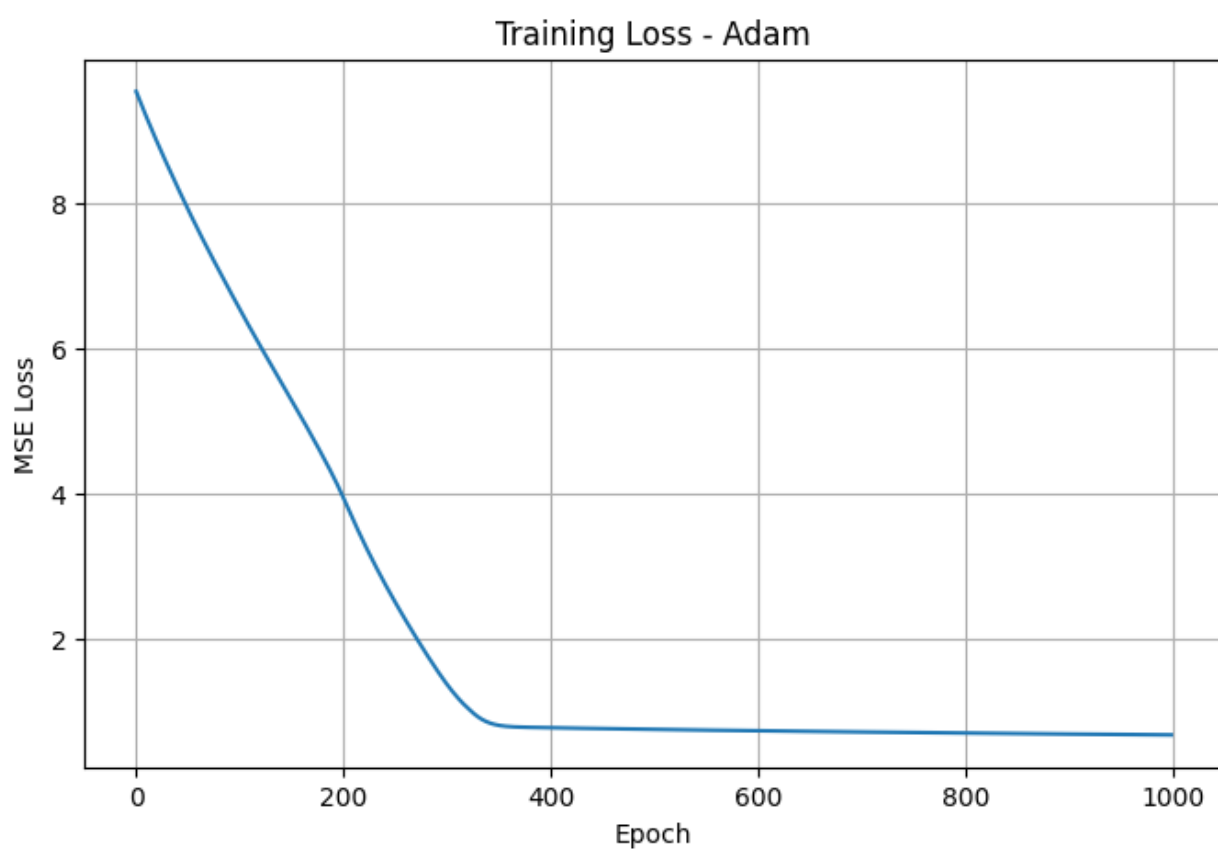


Figure 6: Training loss for Adam.

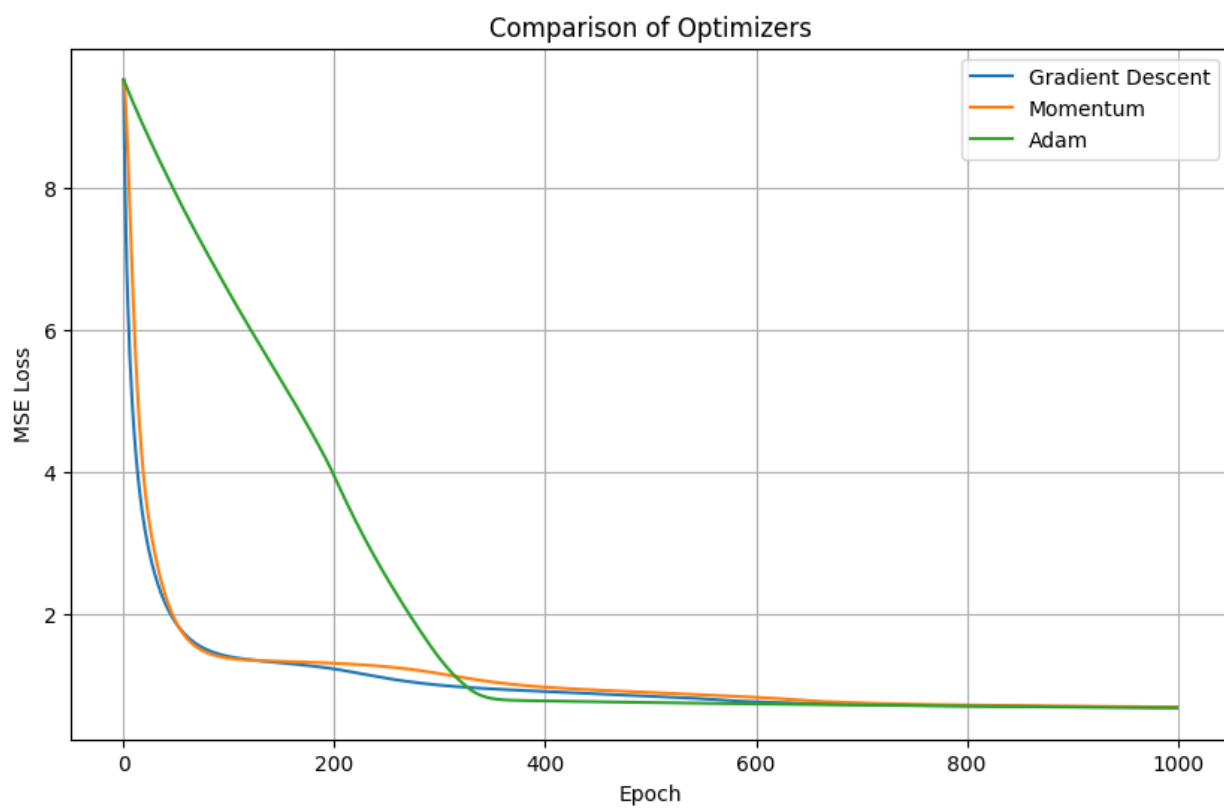


Figure 7: Direct comparison of the three optimizer loss curves.

8. Evaluation and Regression Visualization

The trained models are evaluated on the held-out test set using MSE and RMSE. The notebook also plots predicted values against actual MedHouseVal values. For a perfect predictor, points would lie on the diagonal $y = x$ line. The recorded best-model plot shows the practical prediction spread around this ideal line.

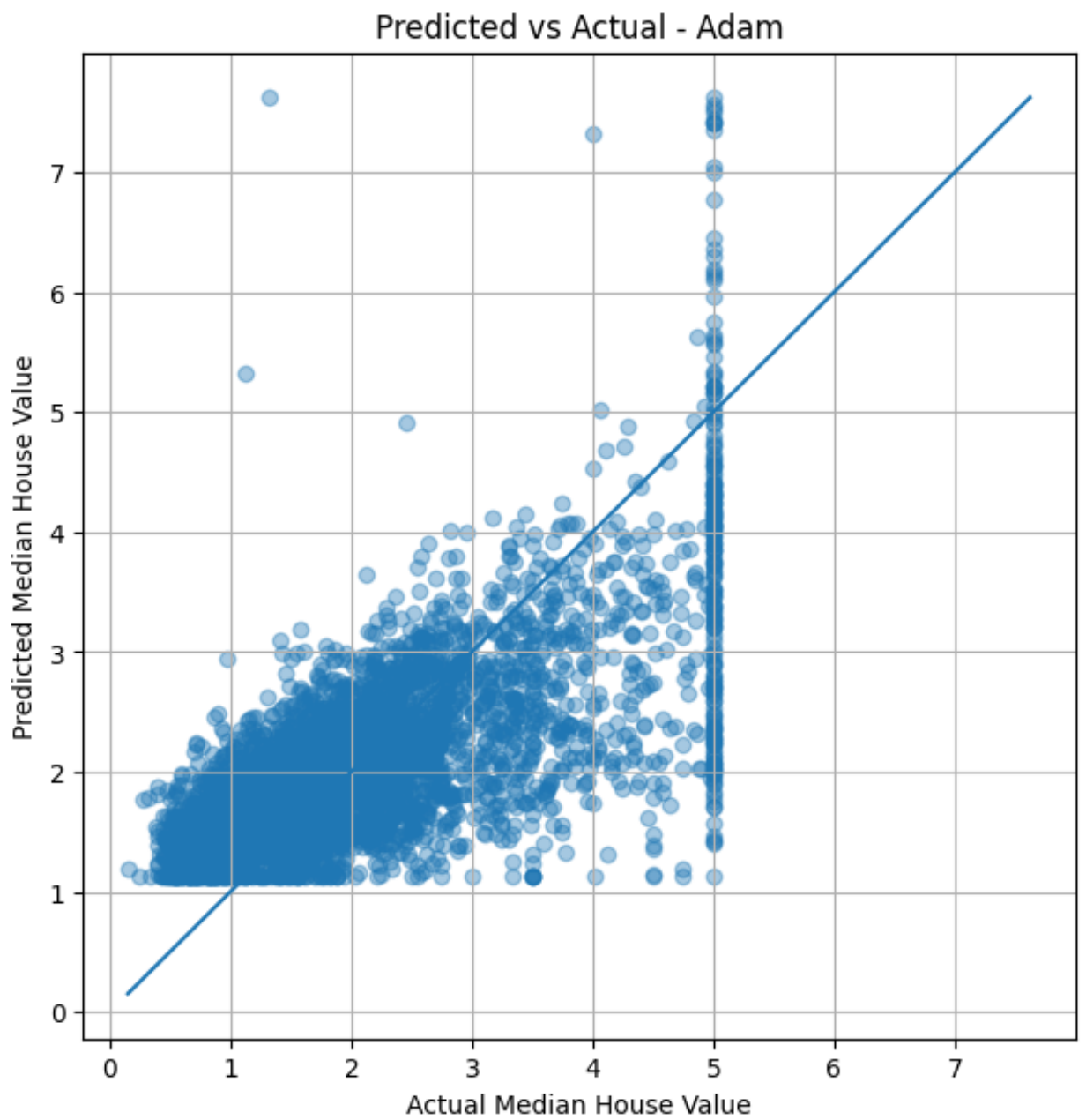


Figure 8: Predicted versus actual house values for the selected best model.

9. Discussion

The experiments show that learning-rate choice has a clear effect on convergence. At 1,000 epochs, the 0.01 learning rate produces much lower training and test loss than 0.001 in the recorded Gradient Descent experiment. This is consistent with the smaller step size requiring more iterations to reach a comparable region of the loss surface.

The optimizer comparison shows that Adam reaches the lowest recorded test error, although its advantage over Basic Gradient Descent is modest. Momentum does not outperform the other two methods under the particular learning rate and 1,000-epoch configuration used here. Therefore, the optimizer should be judged together with its hyperparameters and convergence behavior rather than by optimizer name alone.

The model is intentionally small: only two input variables are supplied to a 5-neuron hidden layer and a 3-neuron hidden layer. Consequently, the model is useful for demonstrating neural-network mechanics, backpropagation, and optimization, but its predictive capability is constrained by using only MedInc and HouseAge rather than all available housing variables.

10. Conclusion

A multi-layer neural network for regression was implemented from scratch using NumPy operations, including ReLU activation, forward propagation, MSE loss, backpropagation, and three parameter-update strategies. The California Housing dataset was split into training and test sets and the two selected features were standardized using training-set statistics.

Across the recorded experiments, Adam with learning rate 0.001 gives the best test performance, with $\text{MSE} = 0.695869$ and $\text{RMSE} = 0.834188$. Basic

Gradient Descent with learning rate 0.01 is a close second, while Momentum is slightly worse under the tested configuration. The learning-rate experiment further demonstrates that 0.01 converges more effectively than 0.001 within the fixed 1,000-epoch budget.

Appendix A — Key Implementation Components

Component	Implementation
Activation	$\text{ReLU}(x) = \max(0, x)$
ReLU derivative	1 for $x > 0$; 0 otherwise
Loss	Mean Squared Error
Architecture	$2 \rightarrow 5 \rightarrow 3 \rightarrow 1$
Training epochs	1000
Optimizers	Gradient Descent, Momentum, Adam
Normalization	Training-set mean and standard deviation
L2 support	Included in the submitted training/backpropagation functions